

1AEL0256

Microcontroladores

Semana 02

Semestre 2026-2

Profesor: Kalun José Lau Gan

Agenda:

- La familia PIC18 de Microchip
- El Curiosity Nano PIC18F47Q10
- Manejo de la documentación (hojas técnicas y repositorios)
- El MPLAB X IDE
- Segundo ejemplo simulado e implementado

Preguntas previas:

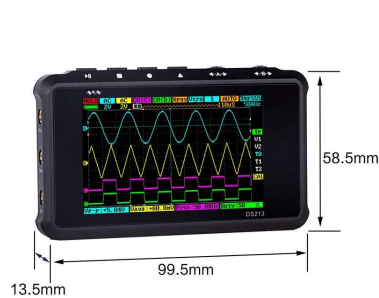
- ¿Debo de tener los materiales para esta semana?
 - Esta semana 2 iniciaremos con las experiencias de laboratorio con el PIC18F47Q10 de manera dirigida, es decir, el profesor hará toda la experiencia de simulación/implementación. Se recomienda tener los materiales para las experiencias de la semana 3 en adelante.
- El lenguaje Assembler para el microcontrolador lo debemos de saber previamente o lo vamos a ver en las clases?
 - Se los va a atender durante las sesiones con el compromiso de que el alumno practique.
- ¿Cuáles son los softwares que necesito instalar en mi PC?
 - MPLABX v6.35, XC8 v4.00
 - Proteus v8.17 en adelante (con soporte de simulación del PIC18F47Q10)
- ¿Qué cable se usa para el Curiosity Nano?
 - Cable USB a MicroUSB de datos (versión antigua)
 - Cable USB a USC-C (versión 2024)

Preguntas previas:

- ¿Hay algún libro sobre este nuevo microcontrolador?
 - No
- Al conectar el Curiosity Nano a la PC solo se enciende, no lo detecta. ¿Qué puede ser?
 - El cable USB que has usado no transporta datos por ende no lo reconoce, cambia de cable.
- ¿Cómo puedo saber si mi PC ha detectado correctamente el Curiosity Nano?
 - Al conectar el Curiosity Nano a la PC, si es que es la primera vez, la PC instalará de manera automática los controladores (drivers) y aparecerá como popup una ventana de explorador de archivos de Windows y se visualizará una unidad de disco correspondiente al Curiosity Nano. (Verificado en Windows 10 y 11)
- ¿El microcontrolador se usa en la industria o solamente con fines académicos? ¿Se puede conectar otro microcontrolador al microcontrolador que estamos usando mediante pines?
 - Los microcontroladores los encuentras en casi todos los equipos electrónicos, desde un reloj hasta el equipo electrónico mas avanzado.
 - Es posible conectar dos microcontroladores y establecer un canal de comunicación entre ellos, mediante el uso de comunicación serial en sus diferentes tipos: I2C, SPI ó UART.

Preguntas previas:

- ¿Es necesario tener un osciloscopio en el curso? ¿Alguna referencia de modelos de scopes pequeños?
 - Es opcional y nos servirá para visualizar señales que cambian de forma en el tiempo, como el PWM.



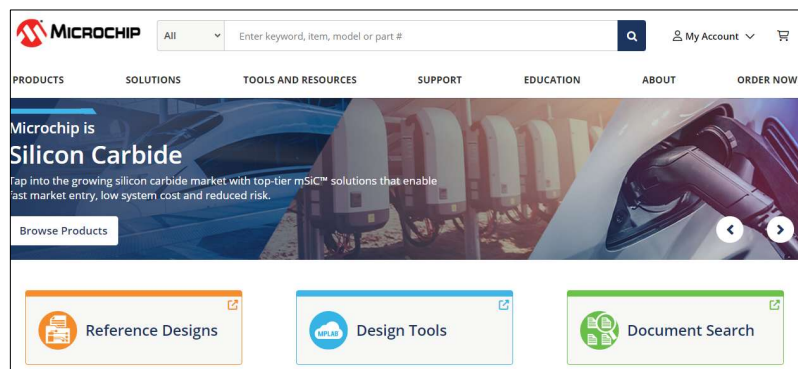
Preguntas previas

- ¿Pueden coexistir diferentes versiones de MPLABX y XC8 en una misma PC?
 - Si, pero de preferencia dejar la versión mas actual solamente a fin de evitar eventuales conflictos.
- Al terminar la instalación del MPLABX me aparece la opción de descargar e instalar el XC8, XC16 y XC32. ¿Cuál de todos selecciono?
 - Solo el XC8
- Los bits de configuración que se modifican después de crear el proyecto. ¿Serán los mismos para cada proyecto que vayamos a realizar?
 - Si, siempre y cuando sea el mismo modelo de microcontrolador, se configuran los mismos 5 bits de configuración para cada proyecto.

Mercado de los microcontroladores

- Actualmente hay preferencias en usar microcontroladores de 8 bits y de 32 bits:
 - Aplicaciones de baja a mediana escala de complejidad -> 8bits
 - Aplicaciones de mediana a alta escala de complejidad -> 32bits
- Se ha disminuido el uso de microcontroladores de 16 bits, muy posible debido a similitud de costos de manufactura y estrecha relación con los de 32 bits.
- Los precios entre micros de 32bits y 8 bits están muy similares.
- Los μC de 8 bits naturalmente consumen menos energía.

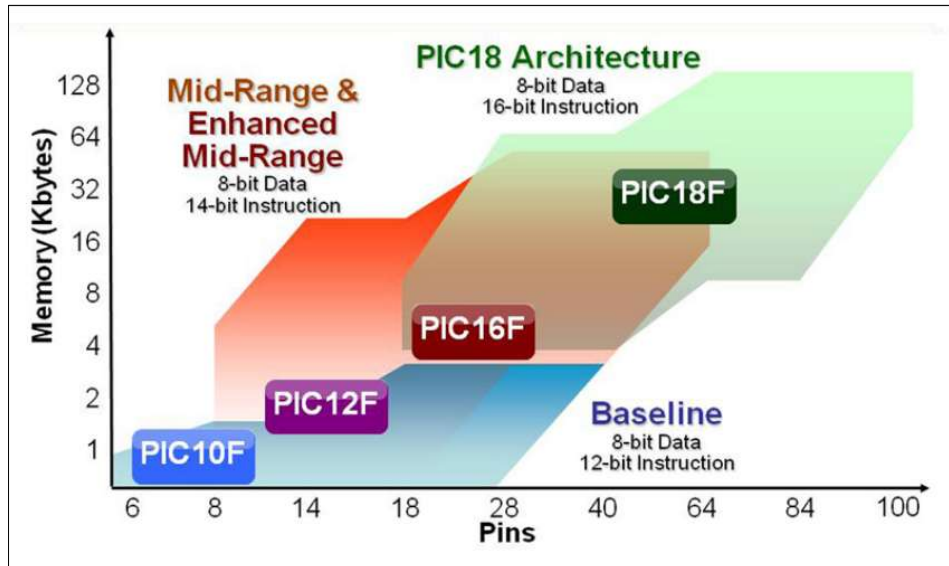
Microchip INC



- División de semiconductores de General Instrument, en 1989 se independizó y se formó Microchip INC.
- Actual líder en el mercado de microcontroladores de **8 bits**.
- En Julio del 2016 compró Atmel y todo su portafolio de productos.
- También adquirió Micrel, Microsemi, KeeLoq, HI-TECH Software, Novocell, ISSC, etc.

Microchip INC

- Familias de microcontroladores PIC de 8 bits



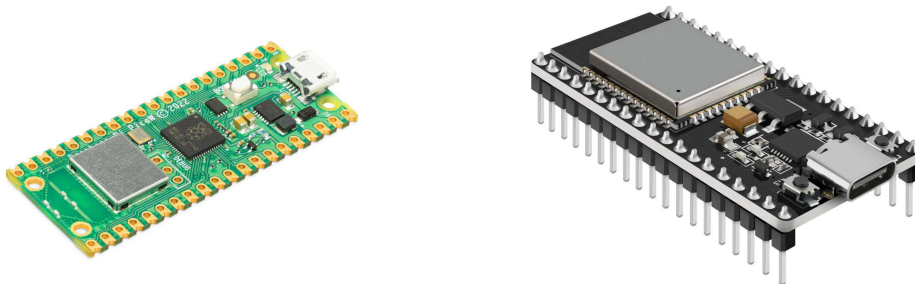
ST Microelectronics

- Líder en microcontroladores de 32bits (STM32)



Nuevos contrincantes en los micros de 32 bits:

- **Raspberry Pi Foundation: RP2040 y RP2350**
 - Costo muy reducidos
 - Flexibilidad
- **Xtensa Tensilica LX7: ESP32**
 - Ampliamente utilizado para aplicaciones de IoT



Familia PIC18 de Microchip

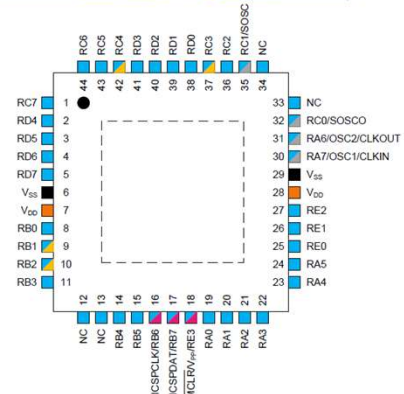
- Primeros dispositivos PIC18: ej. PIC18F452, PIC18F458
- Siguiete generación de PIC18: ej. PIC18F4550
- Actualización de generación de PIC18: Familia K ej. PIC18F45K50
- Familias “Q” de PIC18: Q10, Q40/41, Q43, Q71, Q83/84 ej. PIC18F57Q43
- Nuevas familias Q PIC18: Q20, Q24 (released 27/11/2023)
- Ultimas: Q35 (lanzado en abril del 2026)

[illegible]

El Microcontrolador PIC18F47Q10

Device	Program Memory Flash (bytes)	Data SRAM (bytes)	Data EEPROM (bytes)	I/O Pins	16-bit Timers	Comparators	10-bit ADC ² with Computation (ch)	5-bit DAC	Zero-Cross Detect	CCP/10-bit PWM	CWG	CLC	Low Voltage Detect (LVD)	8-bit TMR with HLT	Windowed Watchdog Timer	CRC with Memory Scan	ELCART	I ² C/SPI	PPS	Peripheral Module Disable	Temperature Indicator	Debug ⁽¹⁾
PIC18F27Q10	128k	3615	1024	25	4	2	24	1	1	2/2	1	8	1	3	Y	Y	2	2	Y	Y	Y	I
PIC18F47Q10	128k	3615	1024	36	4	2	35	1	1	2/2	1	8	1	3	Y	Y	2	2	Y	Y	Y	I

- Encapsulado de 44 pines (el que esta montado en el Curiosity Nano)
- Amplio rango de voltaje de alimentación: 1.8V – 5.5V DC
- Bajo consumo energético (menor a 800nA en modo SLEEP y alrededor de 1mA en operación con 4MHz, 5V)
- Amplio rango de temperatura: -40°C – +85°C (grado industrial)
- Hasta 64MHz de frecuencia en CPU
- Memoria de programa de 128Kbytes y memoria de datos de 3Kbytes
- Periféricos nuevos (detector de cruce por cero, CLC, etc)
- Interrupciones prioritizables
- Módulo PPS para asignación de funciones en los pines



Recordando Arquitectura Harvard

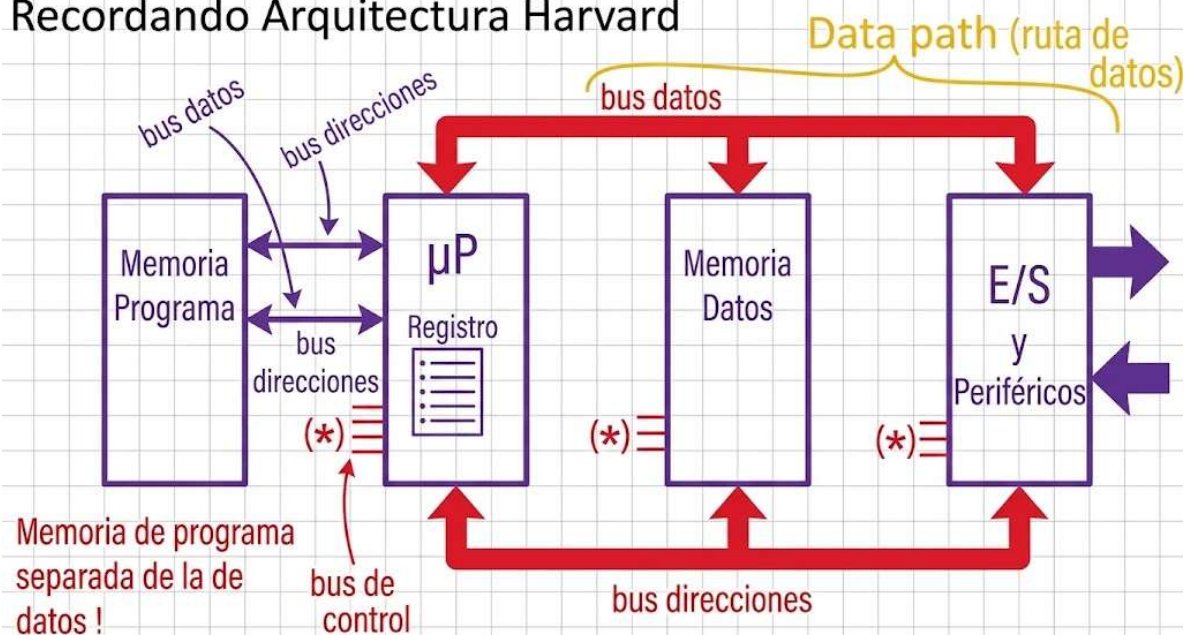
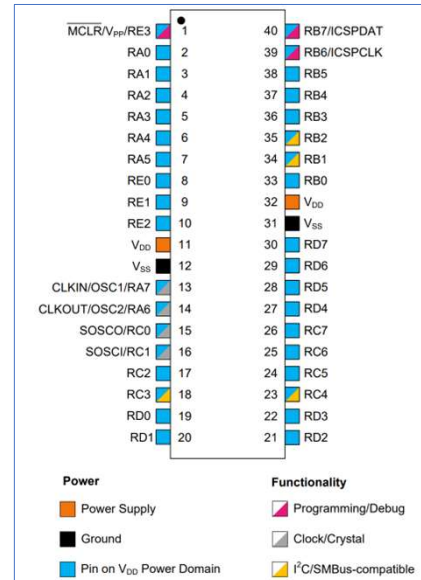


Diagrama de pines del PIC18F47Q10 en formato PDIP 40 pines



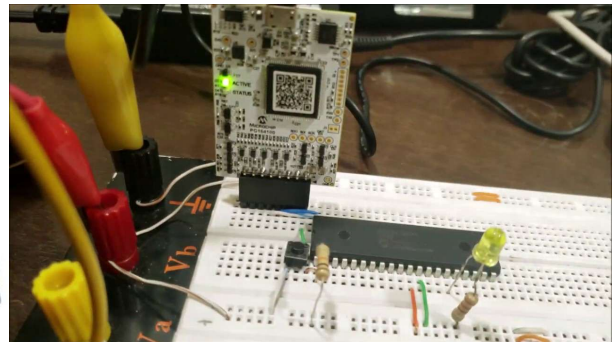
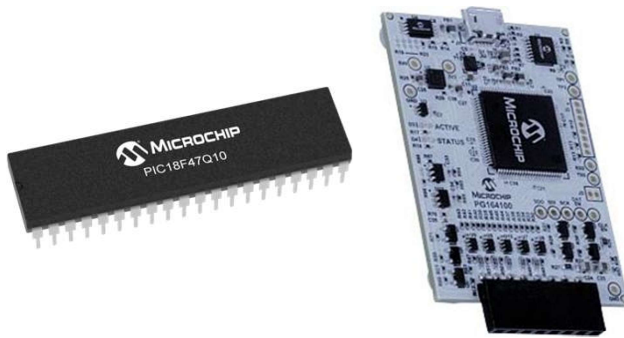
Si es que no se usa el Curiosity Nano

- Necesitamos un microcontrolador en formato DIP para ser usado en el protoboard (ejemplo PIC18F45Q43 ó PIC18F45Q10)
- Necesitamos un programador (ejemplo PICKIT5)



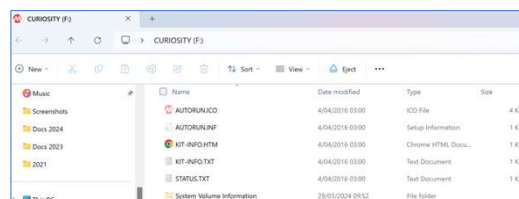
Si es que no se usa el Curiosity Nano

- Necesitamos un microcontrolador en formato DIP para ser usado en el protoboard (ejemplo PIC18F45Q43 ó PIC18F47Q10)
- Necesitamos un programador (ejemplo MPLAB Snap)
- MPLAB Snap Users Guide:
<https://ww1.microchip.com/downloads/aemDocuments/documents/DEV/Product Documents/UserGuides/MPLAB-Snap-In-Circuit-Debugger-User-Guide-50002787.pdf>



Funciones del debugger/programmer del Curiosity Nano PIC18F47Q10

- Proveer energía al módulo (dos líneas)
 - VBUS (conexión directa de la energía del Puerto USB conectado)
 - VTG (conexión de salida de voltaje del sistema ajustable/controlado integrado en el módulo)
- Programar el microcontrolador onboard (el PIC18F47Q10)
- Proveer un puerto serial en la PC para comunicación UART
- Al conectarlo a la PC, aparecerá una unidad de disco llamado "Curiosity"

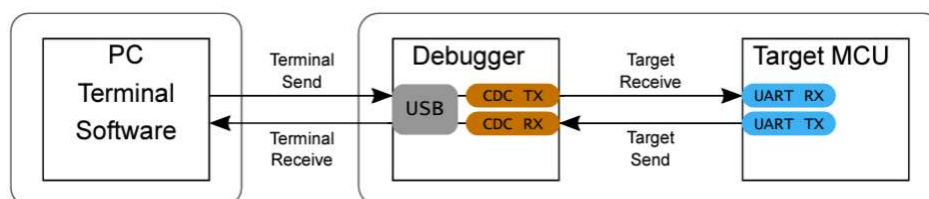


Aclaración sobre la unidad de disco Curiosity

- Al conectar el Curiosity Nano PIC18F47Q10 a la PC aparecerá una unidad de disco llamado "Curiosity".
- Esta unidad de disco **NO DEBE DE USARSE COMO UNIDAD DE ALMACENAMIENTO!, NO ES UNA MEMORIA USB!**
- Esta unidad de disco sirve para hacer drop-off de un firmware

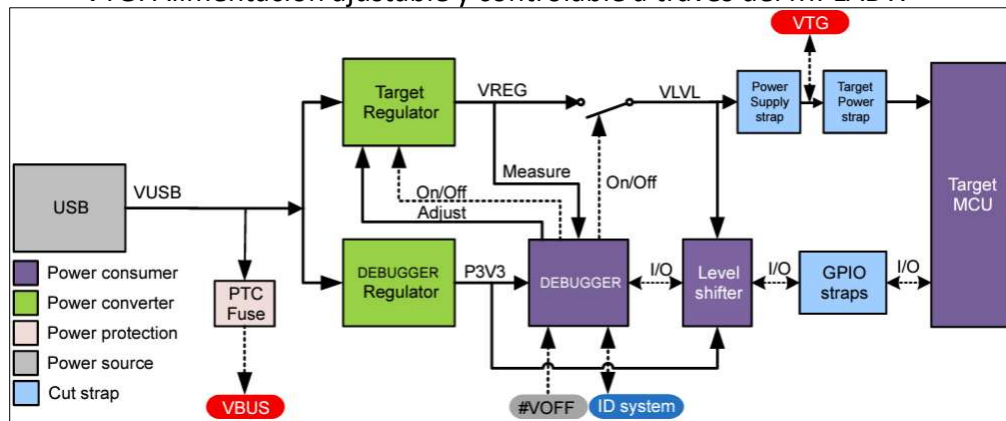
El módulo Curiosity Nano PIC18F47Q10

- Al conectar el Curiosity Nano a la PC con el cable de datos USB-MicroUSB se habilitará un puerto serial para hacer labores de depuración/comunicación
- Dirigirse al Administrador de Dispositivos del Windows (dentro del Panel de Control) para que puedan identificar el puerto serial que se ha habilitado (COMx)
- Software terminal serial puede ser el PuTTY, HyperTerminal, Arduino IDE



El módulo Curiosity Nano PIC18F47Q10

- Posee dos terminales de voltaje de alimentación disponibles para el usuario:
 - VBUS: Conexión directa de la alimentación del puerto USB (5V DC)
 - VTG: Alimentación ajustable y controlable a través del MPLAB X

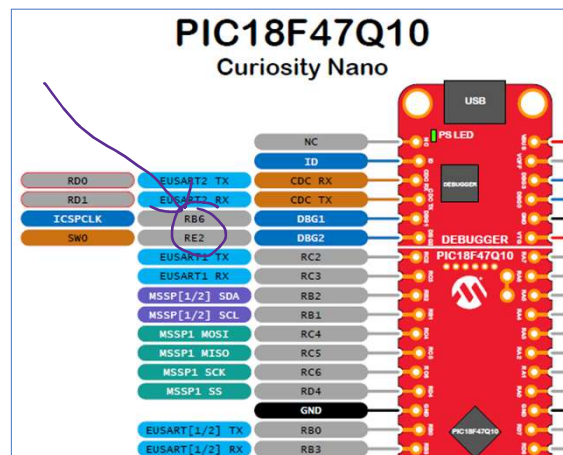


Recomendaciones al usar el Puerto VTG

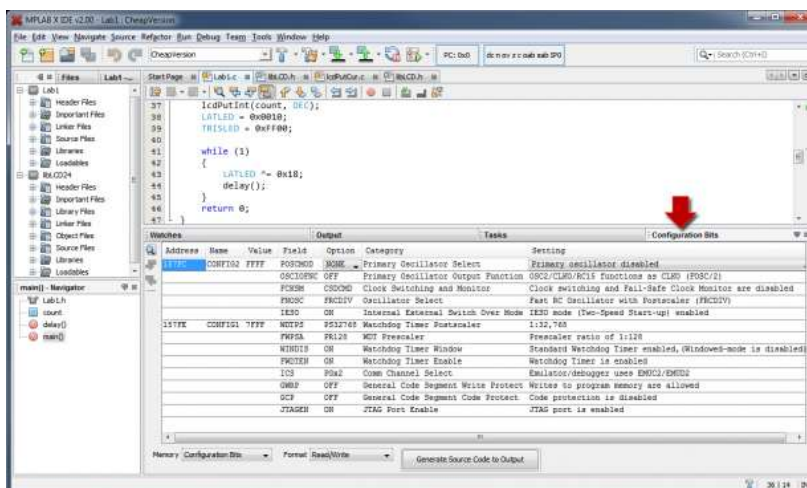
- **NO CONECTAR LOS PINES "VBUS" Y "VTG" A LA VEZ!!!**

- El ajuste de voltaje de este puerto se realiza en el MPLABX
- Antes de grabar el dispositivo en el MPLABX, entrar a propiedades del proyecto y activar la energía (opción Power)

- No encuentro los pines RB6 y RE2 en el Curiosity Nano...
- Los puedes encontrar no rotulados y siguiendo el diagrama de pines en la documentación.



El MPLAB X IDE



- Descargable desde el siguiente link:
<https://www.microchip.com/mplab/mplab-x-ide>
<https://www.microchip.com/development-tools/pic-and-dspic-downloads-archive>

El MPLAB X IDE



- Entorno de desarrollo integrado (Integrated Design Environment - IDE) para los microcontroladores PIC y AVR de Microchip.
- Disponible para Windows, Linux y macOS
- Desarrollo basado en proyecto
- Emplea el compilador XC (XC8, XC16 y XC32), descargable de manera separada.
- Soporte a herramientas de desarrollo propias de Microchip (módulos Curiosity Nano, programadores PICKIT, depuradores ICD, etc)
- Última versión: v6.35

MPASM vs. PICASM (XC8 PIC Assembler)

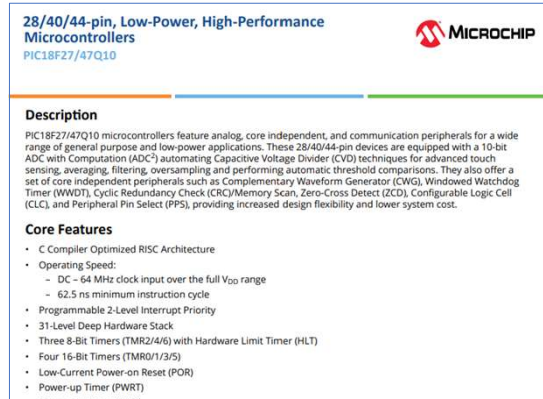
- MPASM fué el lenguaje de programación hasta la v5.35 del MPLABX, actualmente en obsolescencia.
- XC8 PIC Assembler es el nuevo formato de lenguaje y soportado por la nueva versión 6.35
- Las instrucciones en los microcontroladores no han variado, solo la sintaxis de programación (el core del microcontrolador es el mismo).
- Evolución: MPLAB -> MPLAB X (32bits) -> MPLAB X (64bits)
- Documentación: /Program Files/Microchip/xc8/v4.00/docs/MPLAB_XC8_PIC_Assembler_User_Guide.pdf
- Última versión del XC8: v4.00

Consideraciones al momento de instalar el MPLAB X y el XC8

- Tener el sistema operativo en correcto orden (con licencia oficial)
- El MPLAB X IDE y el XC8 se instalan de manera independiente, si solo instalas el MPLAB X IDE no podrás compilar los proyectos.
- No modificar la ruta de instalación de ambos programas, debe de ser instalado en la ruta que te indica por defecto.

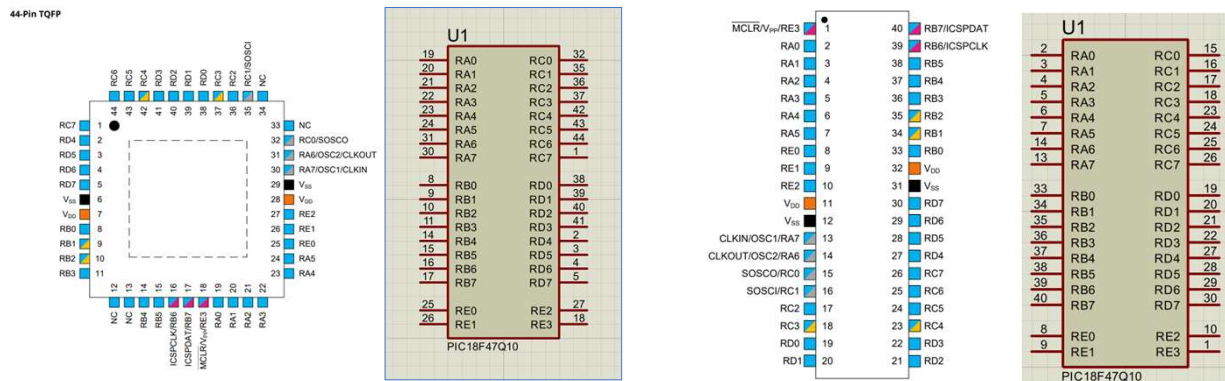
Importancia de tener las hojas técnicas de los IC's a usar:

- Las hojas técnicas (datasheet) son proporcionadas por el fabricante del IC's y se detallan todas las funcionalidades, capacidades, configuraciones, limitaciones, etc de dicho dispositivo, es la información mas fiel.
- En nuestro caso tendremos siempre presente la hoja técnica del microcontrolador PIC18F47Q10 rev.F



Distribución de pines del PIC18F47Q10

- Lo que tenemos en la hoja técnica vs lo que tenemos en Proteus



Revisión de documentos

- Hoja técnica del PIC18F47Q10
 - <https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ProductDocuments/DataSheets/PIC18F27-47Q10-Microcontroller-Data-Sheet-DS40002043.pdf>
- Guía de usuario del Curiosity Nano PIC18F47Q10
 - <https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ProductDocuments/UserGuides/PIC18F47Q10-Curiosity-Nano-Hardware-User-Guide-40002103.pdf>
- Guía de usuario del MPLAB Snap:
 - <https://ww1.microchip.com/downloads/aemDocuments/documents/DEV/ProductDocuments/UserGuides/MPLAB-Snap-In-Circuit-Debugger-User-Guide-50002787.pdf>
- MPLAB XC8 PIC Assembler User Guide:
 - <https://ww1.microchip.com/downloads/aemDocuments/documents/DEV/ProductDocuments/UserGuides/MPLAB-XC8-PIC-Assembler-Users-Guide-DS50002974.pdf>

Importancia de los comentarios en un código fuente

- Cuando uno desarrolla un programa, en cualquier lenguaje de programación, es fundamental colocar comentarios.
- Los comentarios no añaden espacio de memoria luego de la compilación.
- Los comentarios sirven para recordar ideas, configuraciones, procesos, algoritmos, etc que le permitan al programador en un tiempo después ver lo que hizo en dicho momento.
- En MPASM ó XC8 PICASM los comentarios van antecidos por un punto y coma (;)

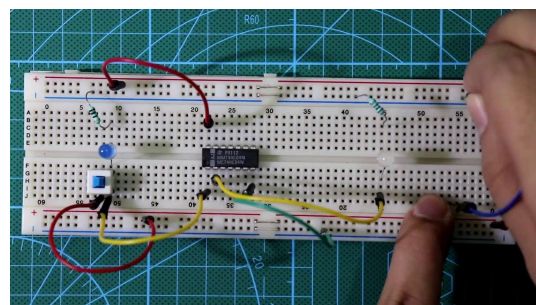
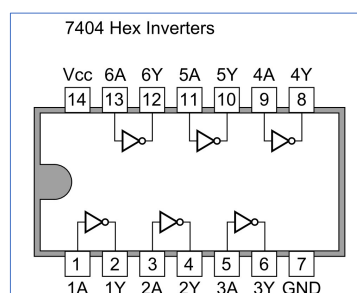
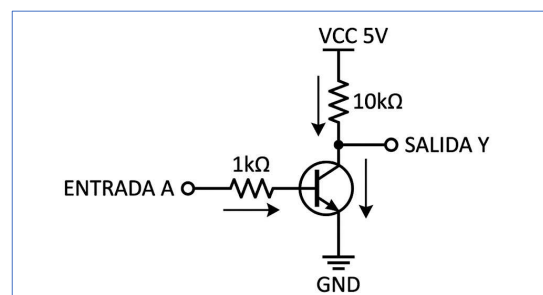
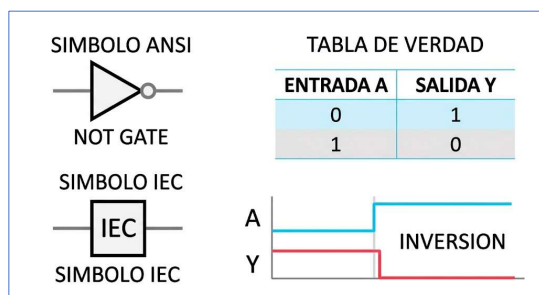
Consideraciones importantes al usar simuladores

- El uso de simuladores ha permitido acelerar los procesos de validación de circuitos eléctricos y electrónicos, **pero** no es un determinante a la hora de validar en forma física.
- En la mayoría de casos en ingeniería electrónica el producto final es algo físico por lo que no solamente podemos fiarnos de una simulación y dar por sentado que la propuesta funcione correctamente.
- En Proteus hay elementos que no se muestran en el momento de hacer simulaciones.
- El software Proteus a la fecha no posee módulo de simulación del microcontrolador PIC18F57Q43 pero si para el PIC18F47Q10

Sobre el MPLAB X IDE

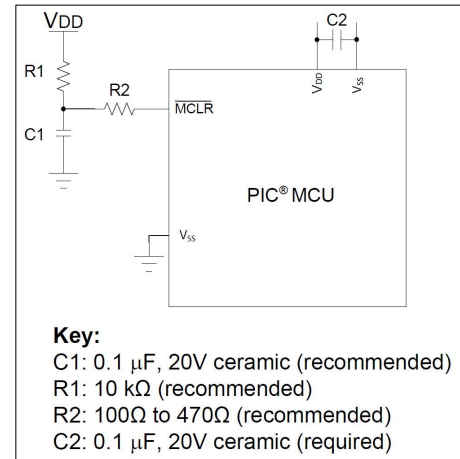
1. Crear un proyecto (seleccionar Standalone Project)
2. Seleccionar el dispositivo microcontrolador (PIC18F47Q10)
3. Seleccionar la herramienta "pic-as" (XC8 PIC Assembler)
4. Crear el archivo header (*.inc) e incluirle los bits de configuración (Window / Target Memory Views / Configuration Bits), dicho archivo debe de estar en la carpeta "Header Files"
5. Crear el archivo fuente (*.s) e incluir el archivo header dentro del cuerpo, dicho archivo debe de estar en la carpeta "Source Files"
6. Para compilar: 
7. El archivo generado de la compilación tiene extensión *.hex ó *.elf
8. Para grabar en el Curiosity Nano: 

El primer ejemplo: una compuerta NOT

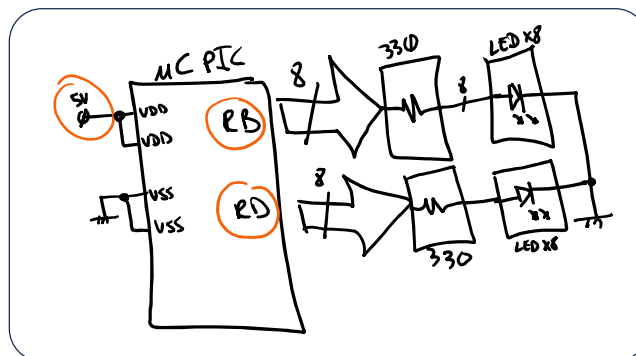


Detalles técnicos iniciales

- Voltaje de alimentación del PIC18F47Q10
 - Voltajes menores a -0.5V son perjudiciales
 - Voltaje de operación máximo 5.5V
 - Voltaje de operación mínimo 1.8V

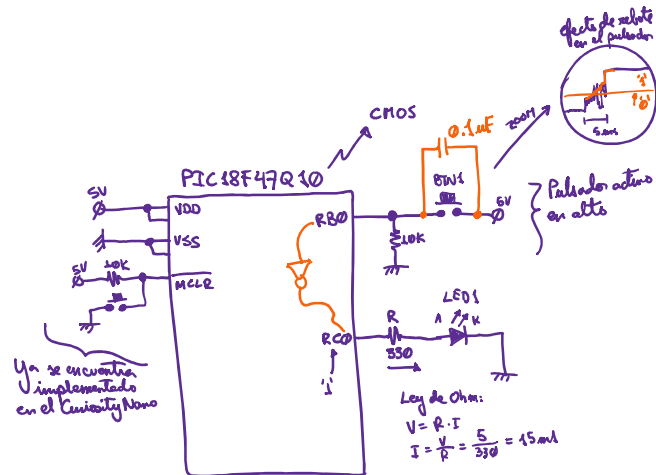
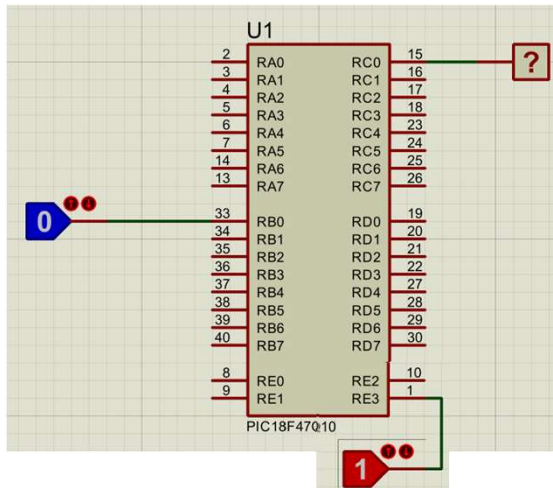


Hay dos pines de Vdd y dos pines de Vss, es necesario conectar todos?

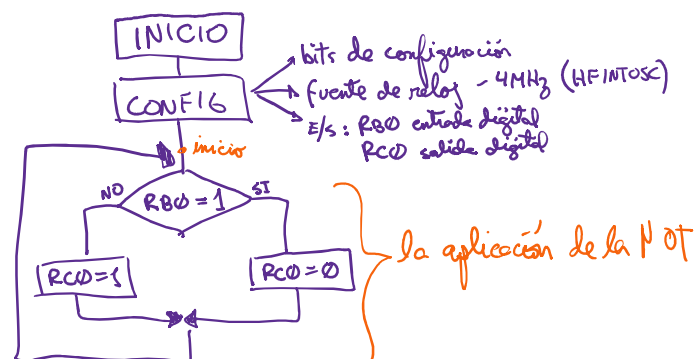


Se deben de conectar todos los pines de alimentación para que el microcontrolador pueda obtener mayor capacidad de corriente en caso lo requiera la aplicación.

Diagrama de simulación en Proteus vs diagrama esquemático electrónico



Algoritmo para que funcione un negador lógico



Plantilla para programas en XC8 PIC Assembler

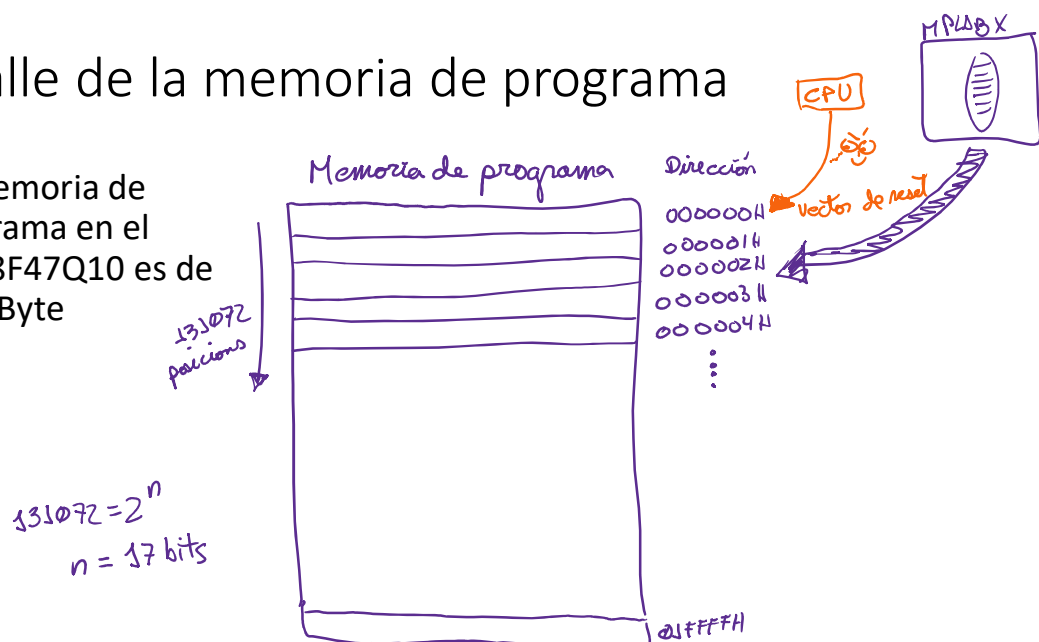
- Tener en cuenta el modelo de microcontrolador a usar, éste debe de coincidir desde la creación del proyecto hasta el detalle de la directiva PROCESSOR

```

1  ;Este es un comentario
2  ;Directiva de procesador
3  PROCESSOR 18F47Q10
4  ;Llamado a la cabecera
5  #include "cabecera.inc"
6
7  ;Declaracion del PSECT
8  PSECT upcino, class=CODE, reloc=2, abs
9  upcino:
10     ORG 000000H      ;direccion del vector de RESET
11     bra configuro    ;salto a label configuro
12
13     ORG 000080H      ;Zona de programa de usuario
14 configuro:
15     ;Configuracion de fuente de reloj
16
17     ;Configuracion de puertos de E/S
18
19 inicio:              ;codigo de la aplicacion
20
21 ;cierre del PSECT
22 end upcino
  
```

Detalle de la memoria de programa

- La memoria de programa en el PIC18F47Q10 es de 128KByte



Desarrollar un negador lógico en un microcontrolador PIC18 Q10 o Q43

Comentarios:

- Tener en cuenta que los puertos de entrada/salida pueden ser entradas o salidas determinado por el registro TRISx, pueden ser analógicos o digitales determinado por el registro ANSELx. Por defecto las E/S son entradas analógicas.
- Según 11.3 de la hoja técnica, los registros de funciones especiales (SFR) se encuentran a partir de la dirección 0E1FH y se extiende hasta 0FFFH, diferente a lo que posee los Q43. Por lo que hay que revisar a detalle en qué banco se encuentran los registros a modificar.
- Ese es el motivo por el cual para acceder a los registros de configuración del oscilador se pasó al Bank14 (BSR=0EH) y para acceder a la manipulación de las E/S se pasó al Bank15 (BSR=0FH)

Desarrollar un negador lógico en un microcontrolador PIC18 Q10 o Q43

Código propuesto en XC8 PIC Assembler:

```

2      ;Directiva de seleccion de procesador
3      PROCESSOR 18F47Q10
4      ;Llamada a la cabecera
5      #include "cabecera.inc"
6
7      ;Declaración del PSECT
8      PSECT upcino, class=CODE, reloc=2, abs
9
10     upcino:
11         ORG 000000H      ;vector de RESET
12         bra configuro    ;salto a label configuro
13
14         ORG 000050H      ;zona de programa de usuario
15     configuro:
16         ;configuracion de la fuente de reloj
17         movlb 0EH        ;me voy al Bank14
18         movlw 60H
19         movwf OSCCON1, 1 ;NOSC=HFINTOSC, NDIV=1:1
20         movlw 02H
21         movwf OSCFREQ, 1 ;HFINTOSC trabaje a 4MHz
22         movlw 40H
23         movwf OSCEN, 1   ;HFINTOSC enabled
24         ;configuracion las E/S
25         movlb 0FH        ;me voy al Bank15
26         bsf TRISA, 0, 1   ;puerto RA0 como entrada
27         bcf ANSELA, 0, 1 ;puerto RA0 como digital
28         bcf TRISC, 0, 1   ;puerto RC0 como salida
29         bcf ANSEL, 0, 1   ;puerto RC0 como digital

```

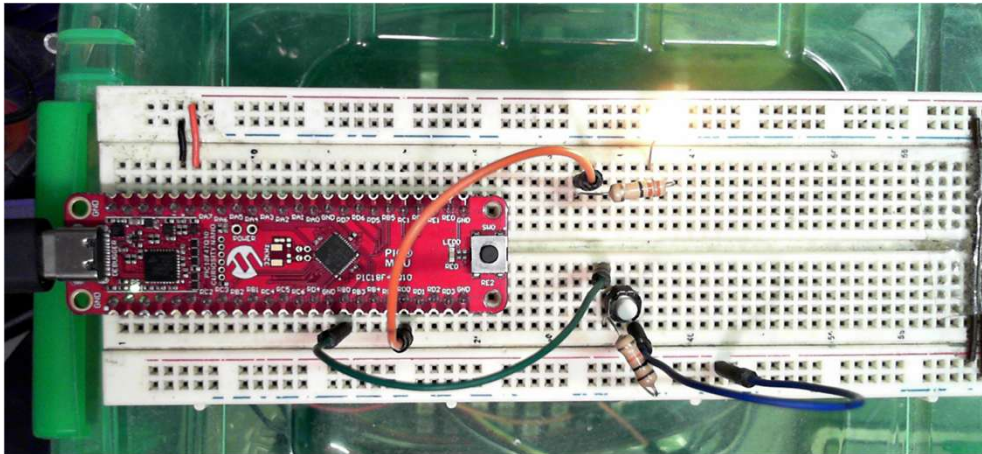
```

30     inicio:      ;el programa de la aplicación
31         btfss PORTA, 0, 1 ;pregunto si RA0 es 1
32         bra falso    ;rpta falsa, salto a label falso
33         bcf LATC, 0, 1 ;rpta verdad, RCO = 0
34         bra inicio    ;salto a label inicio
35     falso:
36         bsf LATC, 0, 1 ;RCO = 1
37         bra inicio    ;salto a label inicio
38         ;Cierre del PSECT
39     end upcino

```

Desarrollar un negador lógico en un microcontrolador PIC18 Q10 o Q43

- Implementación con el Curiosity Nano PIC18F47Q10



Desarrollar un negador lógico en un microcontrolador PIC18 Q10 o Q43

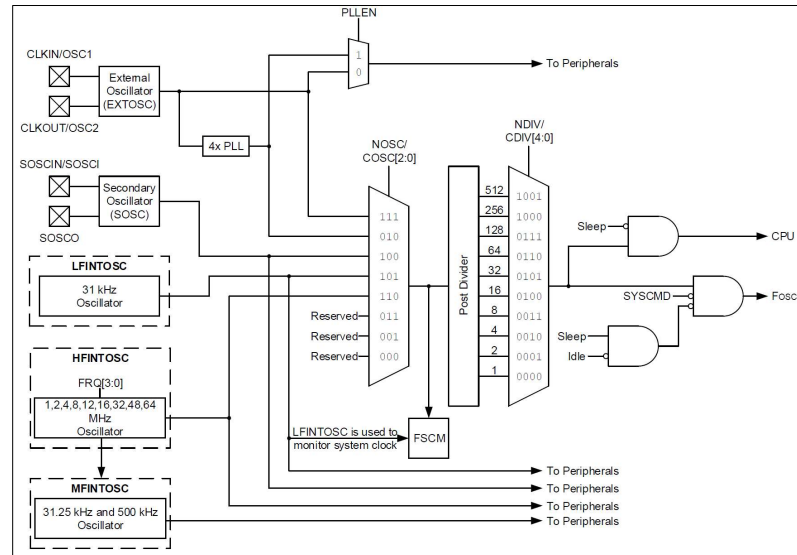
Preguntas:

- Al leer los comentarios del código desarrollado. ¿Tiene relación dicho código con el diagrama de flujo?
- Según lo experimentado. ¿Hay alguna diferencia funcional entre usar el Proteus y la implementación en la vida real?
- ¿A qué frecuencia está trabajando el CPU según lo estipulado en la configuración del módulo de oscilador?
- En el circuito implementado. ¿Por qué no se configuró el weak pull-up en RB0?

Detalles técnicos del módulo de oscilador (fuente de reloj)

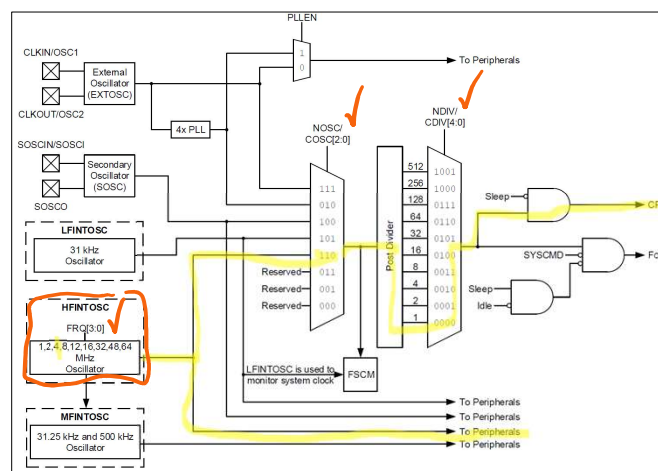
- El módulo de oscilador del PIC18F47Q10 es similar al del PIC18F57Q43

- Oscilador externo (oscilador primario) sirve para que se obtenga la Fuente de reloj de manera externa, ya sea por un cristal o una Fuente de onda cuadrada)
- Oscilador secundario sirve para colocar un cristal de 32.768KHz y hacer aplicaciones de reloj en tiempo real con el Timer1.
- El PLL de este microcontrolador sirve para incrementar la frecuencia de trabajo, hasta 64MHz.
- Usaremos el oscilador interno HFINTOSC para el CPU en todas nuestras aplicaciones.
- MFINTOSC no puede derivarse hacia el CPU, solo hacia periféricos.



Configuración de fuente de reloj con HFINTOSC

Para obtener 4MHz al CPU y periféricos a partir del oscilador interno:



A configurar:

- FRQ
- NOSC/COSC
- NDIV/CDIV

Configuración de fuente de reloj con HFINTOSC

Registros implicados en la configuración del reloj
(PIC18F57Q10):

A configurar:

-FRQ
-NOSC/COSC
-NDIV/CDIV

Address	Name	Bit Pos.	7	6	5	4	3	2	1	0
0x00	Reserved									
...										
0x0ED2										
0x0ED3	OSCCON1	7:0			NOSC[2:0]			NDIV[3:0]		
0x0ED4	OSCCON2	7:0			COSC[2:0]			CDIV[3:0]		
0x0ED5	OSCCON3	7:0	CSWHOLD	SOSCPWR		ORDY	NOSCR			
0x0ED6	OSCCSTAT	7:0	EXTOR	HFOR	MFOR	LFOR	SOR	ADOR		PLL
0x0ED7	OSCCEN	7:0	EXTOEN	HFOEN	MFOEN	LFOEN	SOSCEN	ADOEN		
0x0ED8	OSCTUNE	7:0					HFTUN[5:0]			
0x0ED9	OSCFRQ	7:0						HFFRQ[3:0]		

Configuración de fuente de reloj con HFINTOSC

Registro OSCCON1:

12.5.2 OSCCON1

Name: OSCCON1
Address: 0x0ED3

Oscillator Control Register 1

Bit	7	6	5	4	3	2	1	0
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	f	f	f	q	q	q	q	q

Bits 6:4 – NOSC[2:0] New Oscillator Source Request^(1,2,3)
Requests a new oscillator source per the [NOSC/COSC Clock Source Selection Table](#).

Bits 3:0 – NDIV[3:0] New Divider Selection Request
Requests the new postscale division ratio per the [NDIV/CDIV Clock Divider Selection Table](#).

Notes:

- The default value is determined by the RSTOSC Configuration bits. See the Reset Oscillator (RSTOSC) selection table for the RSTOSC selections.
- If NOSC is written with a reserved value, the operation is ignored and neither NOSC nor NDIV is written.
- When CSWEN = 0, these bits are read-only and cannot be changed from the RSTOSC value.

Table 12-2. NOSC/COSC Clock Source Selection Table

NOSC / COSC	Clock Source
111	EXTOSC ⁽¹⁾
110	HFINTOSC ⁽²⁾
101	LFINTOSC
100	SOSC
011	Reserved
010	EXTOSC + 4xPLL ⁽³⁾
001	Reserved
000	Reserved

Table 12-3. NDIV/CDIV Clock Divider Selection Table

NDIV / CDIV	Clock Divider
1111-1010	Reserved
1001	512
1000	256
0111	128
0110	64
0101	32
0100	16
0011	8
0010	4
0001	2
0000	1

Configuración de fuente de reloj con HFINTOSC

Registro OSCFRQ:

12.5.6 OSCFRQ

Name: OSCFRQ
Address: 0x0011

HFINTOSC Frequency Selection Register

Bit 7 6 5 4 3 2 1 0

Access: R/W R/W R/W R/W R/W R/W R/W R/W
Reset: 0 0 0 0 0 0 0 0

FRQ[3:0]

Bits 3:0 – FRQ[3:0] HFINTOSC Frequency Selection

FRQ	Nominal Freq (MHz)
1111-1001	Reserved
1000	64
0111	48
0110	32
0101	16
0100	12
0011	8
0010	4
0001	2
0000	1

Configuración de fuente de reloj con HFINTOSC

Registro OSCEN:

12.5.8 OSCEN

Name: OSCEN
Address: 0x0000

Oscillator Enable Register

Bit 7 6 5 4 3 2 1 0

Access: R/W R/W R/W R/W R/W R/W R/W R/W
Reset: 0 0 0 0 0 0 0 0

EXTOEN HFOEN MFOEN LFOEN SOSSEN ADOEN PLEN

Bit 7 – EXTOEN External Oscillator Enable

Value	Description
1	EXTOSC is explicitly enabled, operating as specified by FEXTOSC
0	EXTOSC can be enabled by a peripheral request

Bit 6 – HFOEN HFINTOSC Enable

Value	Description
1	HFINTOSC is explicitly enabled, operating as specified by OSCFRQ
0	HFINTOSC can be enabled by a peripheral request

Bit 5 – MFOEN MFINTOSC Enable

Value	Description
1	MFINTOSC is explicitly enabled
0	MFINTOSC can be enabled by a peripheral request

Bit 4 – LFOEN LFINTOSC Enable

Value	Description
1	LFINTOSC is explicitly enabled
0	LFINTOSC can be enabled by a peripheral request

Bit 3 – SOSSEN Secondary Oscillator Enable

Value	Description
1	SOSC is explicitly enabled, operating as specified by SOSCPWR
0	SOSC can be enabled by a peripheral request

Bit 2 – ADOEN ADCRC Oscillator Enable

Value	Description
1	ADCR is explicitly enabled
0	ADCR may be enabled by a peripheral request

Bit 0 – PLEN PLL Enable⁽¹⁾

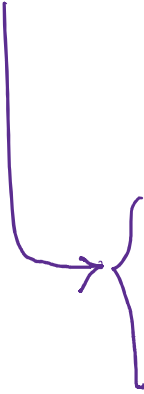
Value	Description
1	EXTOSC multiplied by the 4x system PLL is used by a peripheral request
0	EXTOSC is used by a peripheral request

Note:

1. This bit only controls external clock source supplied to the peripherals and has no effect on the system clock.

Configuración de fuente de reloj con HFINTOSC

- Configuraremos el PIC18F47Q10 para que funcione a 4MHz con el HFINTOSC



```

15      ;configuracion de la fuente de reloj
16      movlb 0EH          ;me voy al Bank14
17      movlw 60H
18      movwf OSCCON1, 1   ;NOSC=HFINTOSC, NDIV=1:1
19      movlw 02H
20      movwf OSCFRQ, 1    ;HFINTOSC trabaje a 4MHz
21      movlw 40H
22      movwf OSCEN, 1     ;HFINTOSC enabled
  
```

Configuración de fuente de reloj con HFINTOSC

- ¿Por qué siempre se trabaja con 4MHz y no mas rápido?¿Qué pasaría si trabajamos siempre a 32MHz?
 - Porque trabajar a mayor frecuencia el microcontrolador consumirá mas energía.
 - El microcontrolador trabajará mas rápido
 - Se trabaja a 4MHz debido a que cada instrucción en ensamblador se ejecuta a 1us.

Ejercicios propuestos:

- Modificar el circuito y diagrama de flujo para que con un pulsador se cambie el periodo de titileo entre 500ms y 250ms.
- Basado en el punto anterior y usando el mismo pulsador intercambiar el periodo de titileo entre 500ms, 250ms, 100ms y 50ms.
- Efecto del auto fantástico (corrimiento de iluminación de ocho LEDs)

Fin de la sesión!